

Daily Technical Report

Project Title	<i>IoT Training Program</i>
Report / Day No.	<i>Day 1</i>
Date	<i>19/7/2026</i>
Prepared By	Malak Alsharqawi
Organization	<i>Cyber Robot</i>

Table of Contents

Table of Contents	2
1. Daily Overview	3
1.1 Summary	3
1.2 Tasks Completed Today	3
Task 1:	3
1. Objective	3
2. Procedure.....	3
<i>Step 1 — Install the CP210x USB-to-UART driver</i>	3
<i>Step 2 — Install Windows Subsystem for Linux (WSL)</i>	3
<i>Step 3 — Install and configure PlatformIO</i>	3
<i>Step 4 — Configure USB/serial passthrough (usbipd)</i>	3
3. Code / Configuration.....	4
4. Schematic.....	4
5. Photos	5
6. Issues & Troubleshooting.....	5
7. Results & Outcome	5
Task 2: GitHub	5
1. Objective	5
2. Procedure.....	5
<i>Phase 1 — SSH key setup</i>	5
<i>Phase 2 — Initialize and link the repository</i>	5
<i>Phase 3 — Push to GitHub</i>	6
3. Code / Configuration.....	6
4. Issues & Troubleshooting.....	6
5. Results & Outcome	6
3. Project Documentation & Notes	6

1. Daily Overview

1.1 Summary

This session successfully set up a complete ESP32 development environment by linking Windows hardware to a Linux-based WSL backend for building and flashing firmware. The environment was fully verified by running an external LED blink sketch and establishing a secure, password-free connection to GitHub using SSH.

1.2 Tasks Completed Today

- VS Code & PlatformIO Setup + ESP32 Blink
- Git & Github

Task 1: VS Code & PlatformIO Setup + ESP32 Blink

1. Objective

Set up a complete embedded development environment capable of recognizing, building for, and flashing firmware to an ESP32 microcontroller from a Linux-based toolchain running on Windows.

2. Procedure

Step 1 — Install the CP210x USB-to-UART driver

- Downloaded and installed the CP210x VCP driver so Windows recognizes the ESP32 over USB.

Step 2 — Install Windows Subsystem for Linux (WSL)

- Ran the one-command installation and restarted the machine to apply virtualization layers.
- Initialized the Linux user account (username and password) on first boot.
- Verified the installed distribution and WSL version using the status list command.
- Updated the internal package database before installing any compilers or extensions.

Step 3 — Install and configure PlatformIO

- Installed the PlatformIO IDE extension inside VS Code (connected to the WSL: Ubuntu backend).
- Resolved a Python interpreter error by installing python3 and python3-venv inside WSL, then retried the extension setup.

Step 4 — Configure USB/serial passthrough (usbipd)

- Granted the Linux user permission to access serial devices (dialout group).
- Force-bound the ESP32's USB Bus ID on the Windows host using usbipd.
- Attached the device to the WSL distribution with auto-attach enabled for persistent reconnection.

3. Code / Configuration

Key commands used during setup:

```
# Install WSL
wsl --install

# Verify installation
wsl --list -v

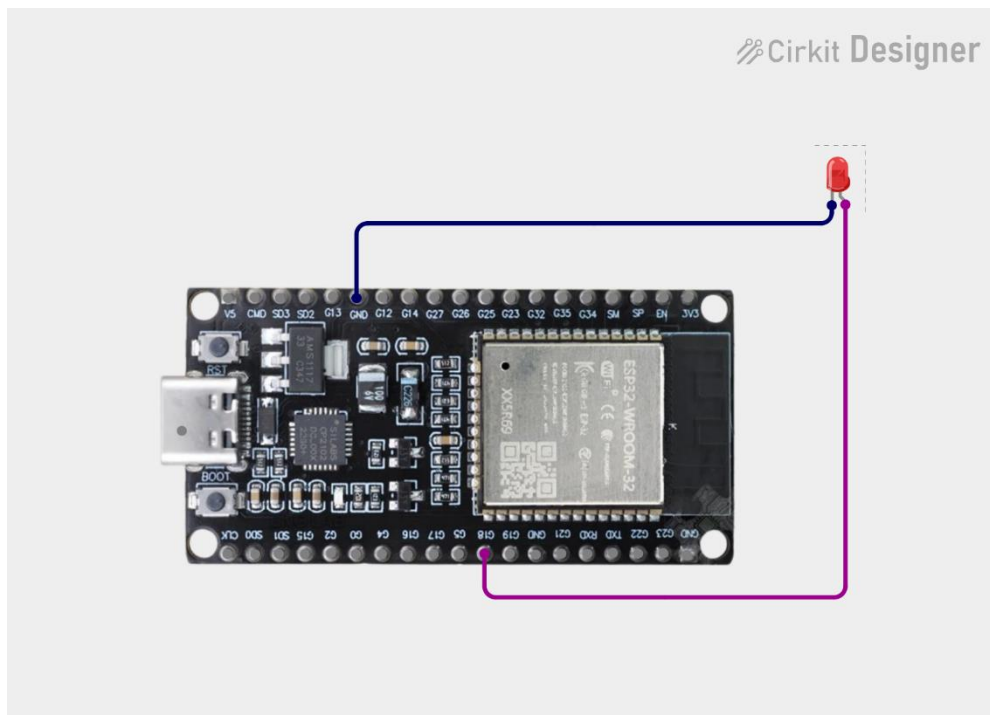
# Update Linux packages
sudo apt update && sudo apt upgrade -y

# Fix PlatformIO Python interpreter error
sudo apt update && sudo apt install python3 python3-venv -y

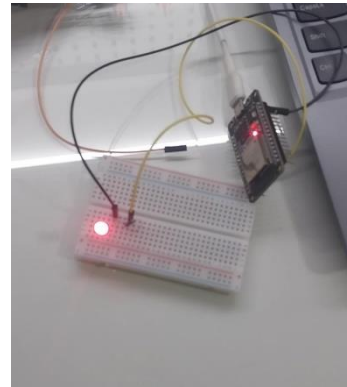
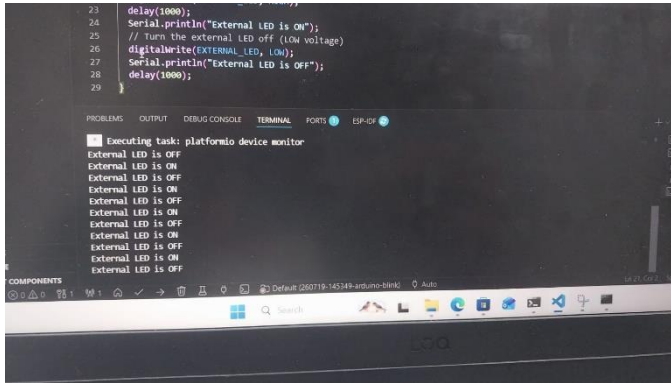
# Grant serial device permissions
sudo usermod -a -G dialout $USER

# Bind and attach ESP32 (run in Windows PowerShell as Administrator)
usbipd list
usbipd bind --busid <YOUR_BUS_ID> --force
usbipd attach --wsl --busid <YOUR_BUS_ID> --auto-attach
```

4. Schematic



5. Photos



6. Issues & Troubleshooting

PlatformIO initially failed with “Can not find working Python 3.6+ Interpreter” because it was searching for Python inside the WSL Linux environment rather than Windows. Resolved by installing python3 and python3-venv inside WSL and retrying.

7. Results & Outcome

The ESP32 was successfully recognized by WSL over a shared USB/serial connection, PlatformIO built and uploaded a basic Blink LED sketch (GPIO 18, 115200 baud), and serial monitor output was confirmed to display “External LED is ON/OFF” at 1-second intervals.

Task 2: GitHub

1. Objective

Turn an existing local project folder into a version-controlled Git repository and link it to a remote GitHub repository using a secure SSH connection.

2. Procedure

Phase 1 — SSH key setup

- Generated a new ed25519 SSH key pair and added the public key to GitHub (Settings → SSH and GPG keys).

Phase 2 — Initialize and link the repository

- Navigated to the project directory and ran git init to start version tracking.
- Staged and committed all existing files as the initial commit.

- Renamed the default branch to main and added the GitHub remote using the SSH URL.

Phase 3 — Push to GitHub

- Pushed the local commit history to GitHub and set the upstream tracking branch.
- Accepted GitHub's host fingerprint on first connection.

3. Code / Configuration

Commands executed:

```
# Generate SSH key
ssh-keygen -t ed25519 -C "your_email@example.com"
cat ~/.ssh/id_ed25519.pub    # copy this into GitHub

# Initialize and commit
cd ~/test/IoT_training
git init
git add .
git commit -m "Initial commit of existing project files"
git branch -M main
git remote add origin git@github.com:<user>/<repo>.git

# Push
git push -u origin main
```

4. Issues & Troubleshooting

None — first-time SSH host verification prompt was accepted as expected.

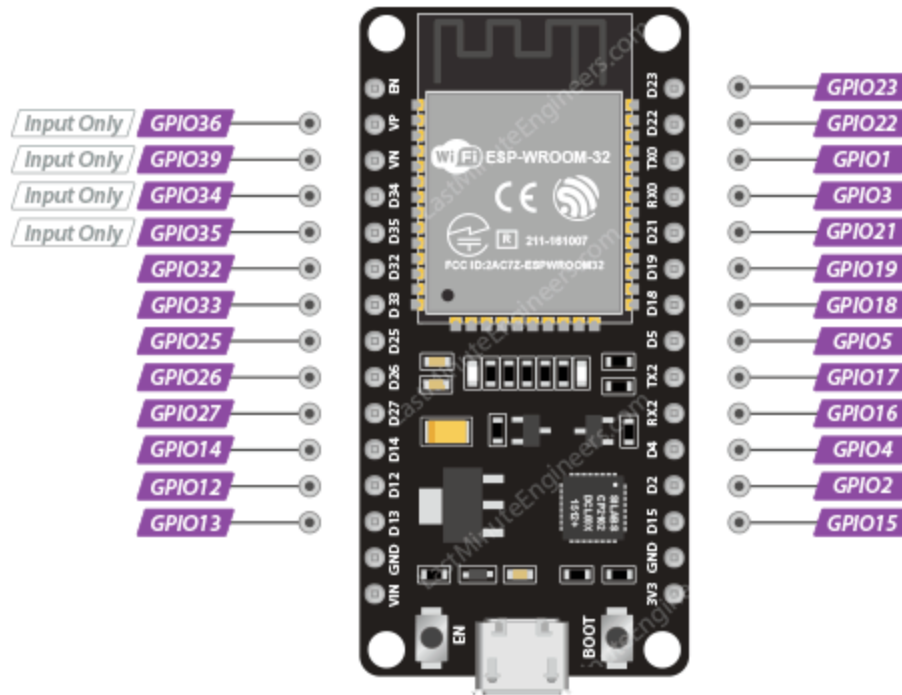
5. Results & Outcome

The local project was successfully version-controlled and pushed to the remote GitHub repository over SSH; subsequent commits can now be pushed directly with git push.

3. Project Documentation & Notes

- **Development Notes & Key Learnings** All daily progression logs, troubleshooting steps, and core takeaways for this initial project milestone are actively documented here: [📖 View My First-Day Notion Notebook](#)

- **Hardware Schematic & Circuit Layout** The system architecture design, GPIO pinout mapping, and electrical connections were mapped out using the cloud platform: [👉 Open Circuit Designer Workspace](#)
- **GPIO Pinout**



End of Report